JAVA WEB

Tutorial da Aula 02

CRUD de alunos em memória com Servlets e JSP

GET	POST	CRUD

Continuação: PrimeiraAplicacaoWeb da Aula 01
Ambiente: NetBeans + Ant + JDK 17 ou superior + Tomcat 11
Sem Maven e sem banco de dados

Inclui imagens orientativas, códigos completos, explicações e exercícios.

1. O que será construído

A Aula 02 transforma o cadastro simples da primeira aula em um pequeno sistema de gerenciamento de alunos. O sistema manterá os objetos em memória enquanto o Tomcat estiver executando.

Objetivos

- Aprofundar o uso de doGet e doPost.
- Implementar inclusão, consulta, edição e exclusão.
- Separar modelo, repositório, controle e visualização.
- Entender forward, sendRedirect e o padrão Post/Redirect/Get.
- Proteger páginas JSP dentro de WEB-INF.
- Validar dados no servidor.

Funcionalidades

Operação	Requisição	Resultado
Listar	GET /alunos	Exibe todos os alunos.
Novo	GET /alunos?acao=novo	Abre o formulário vazio.
Editar	GET /alunos?acao=editar&id;=1	Abre o formulário preenchido.
Salvar	POST /alunos	Inclui ou atualiza.
Excluir	POST /alunos com acao=excluir	Remove e redireciona.

2. Conceitos da continuação

CRUD

CRUD representa Create, Read, Update e Delete. Neste projeto: cadastrar corresponde a Create; listar e localizar correspondem a Read; editar corresponde a Update; excluir corresponde a Delete.

Forward e redirect

Recurso	Comportamento	Uso nesta aula
forward	Continua no servidor, preserva request e não altera a URL.	Abrir JSP e reenviar formulário com erro.
sendRedirect	Envia nova resposta ao navegador, que faz outro GET.	Após salvar ou excluir.

Post/Redirect/Get

Depois de um POST válido, o Servlet redireciona para GET /alunos. Assim, atualizar a página não repete o cadastro nem a exclusão.

Repositório em memória

O repositório simula a camada de acesso a dados usando um Map. Ele permite aprender o fluxo da aplicação antes da introdução de JDBC. Todos os dados são perdidos ao reiniciar ou republicar o projeto.

3. Preparação do projeto

Continue no projeto PrimeiraAplicacaoWeb. Antes de alterar arquivos, faça uma cópia de segurança da pasta do projeto.

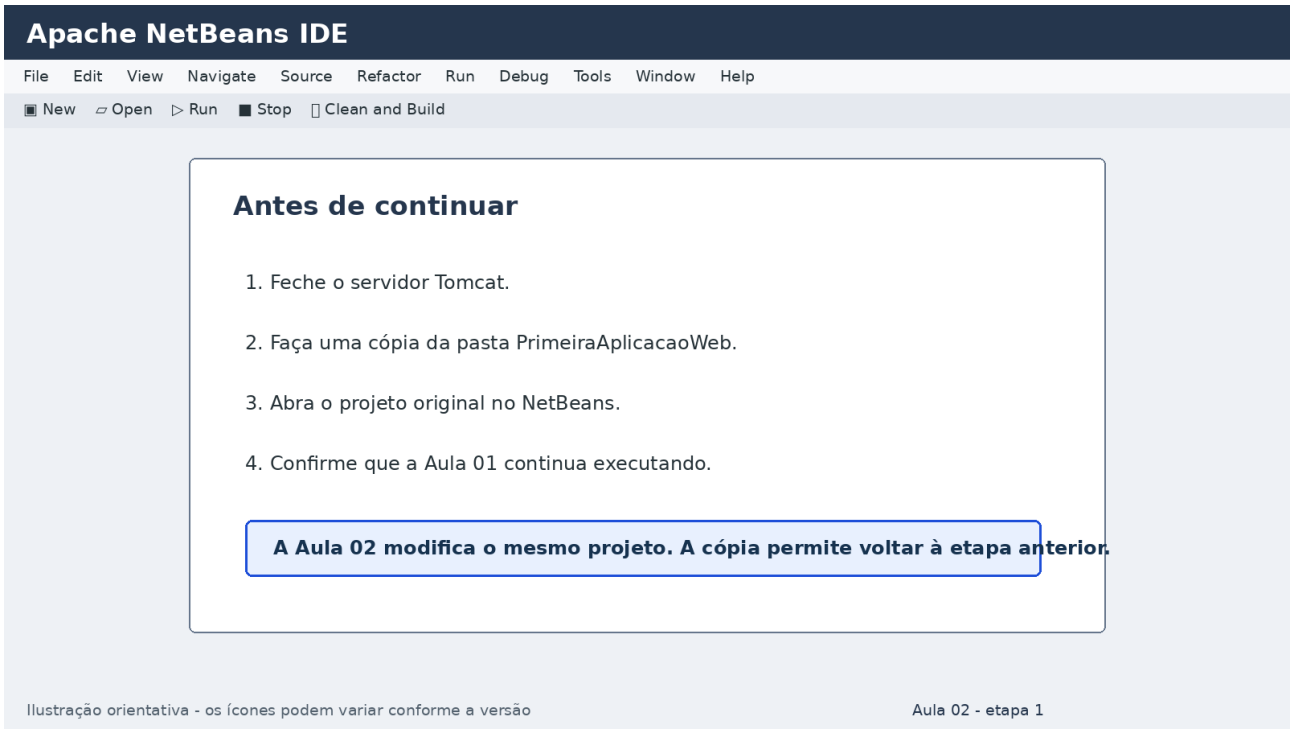


Figura 1 - Preparação segura antes de modificar a Aula 01.

A classe Pessoa e o CadastroServlet da primeira aula podem permanecer no projeto, mas deixarão de ser a entrada principal. A nova aplicação usará Aluno, AlunoRepositorio e AlunoServlet.

4. Criar os novos pacotes

Em Source Packages, crie os pacotes `br.edu.exemplo.repositorio` e, se necessário, confirme os pacotes de modelo e controle.

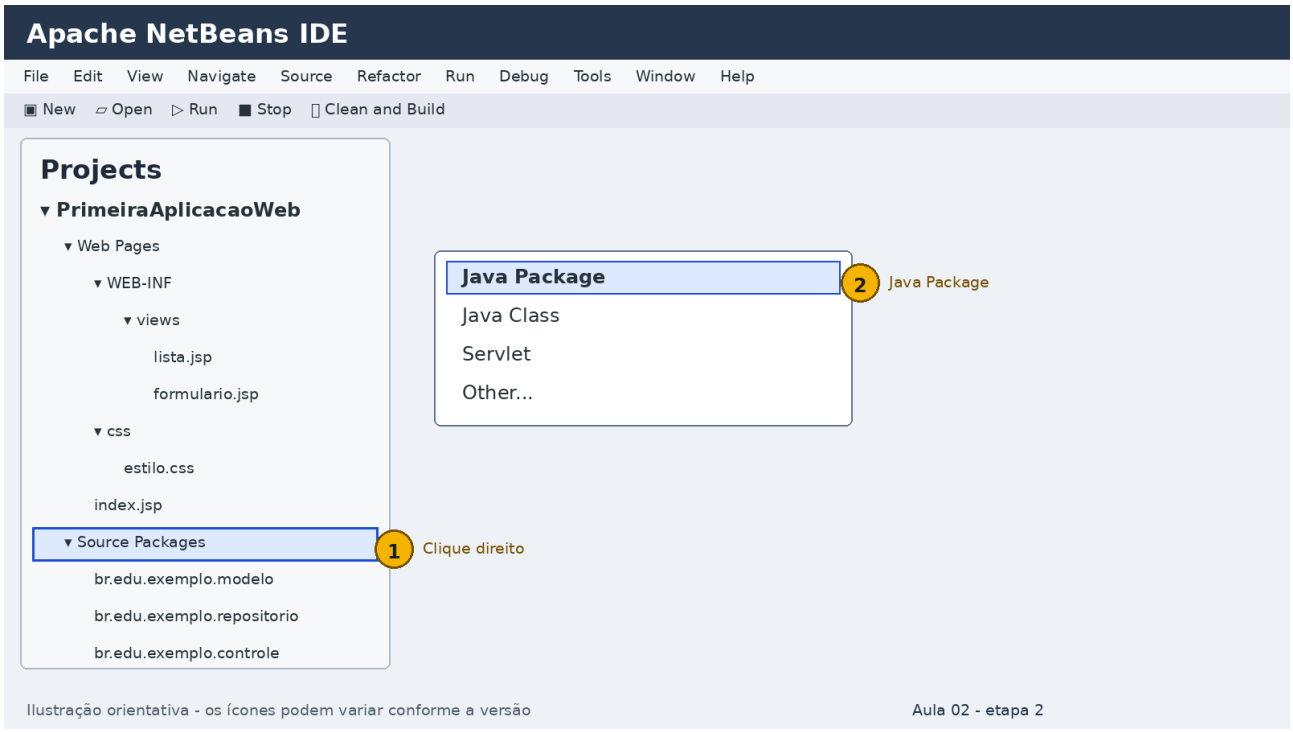


Figura 2 - Criando um pacote em Source Packages.

Pacotes utilizados

Pacote	Responsabilidade
br.edu.exemplo.modelo	Objetos do domínio, como Aluno.
br.edu.exemplo.repositorio	Armazenamento e operações sobre alunos.
br.edu.exemplo.controle	Servlet que recebe requisições e decide o fluxo.

5. Classe Aluno

No pacote `br.edu.exemplo.modelo`, crie a classe Aluno.

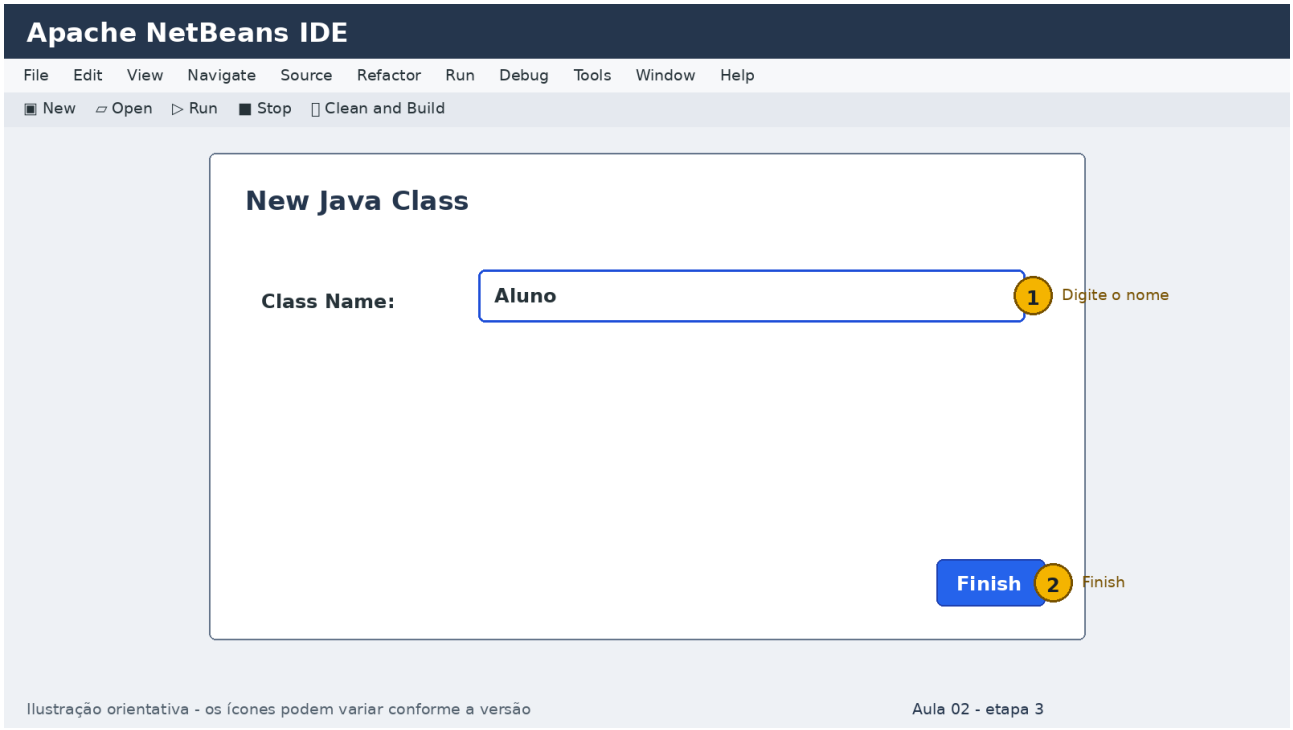


Figura 3 - Criando Aluno.java.

Código completo de Aluno.java

```
package br.edu.exemplo.modelo;

public class Aluno {

    private int id;
    private String nome;
    private String email;
    private String curso;
    private int semestre;
    private String turno;

    public Aluno() {
    }

    public Aluno(int id, String nome, String email, String curso,
                 int semestre, String turno) {
        this.id = id;
        this.nome = nome;
        this.email = email;
        this.curso = curso;
        this.semestre = semestre;
        this.turno = turno;
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNome() {
        return nome;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getCurso() {
        return curso;
    }

    public void setCurso(String curso) {
        this.curso = curso;
    }

    public int getSemestre() {
        return semestre;
    }

    public void setSemestre(int semestre) {
        this.semestre = semestre;
    }

    public String getTurno() {
        return turno;
    }

    public void setTurno(String turno) {
        this.turno = turno;
    }
}
```

O identificador será atribuído pelo repositório. O valor zero representa um objeto ainda não cadastrado.

6. Repositório em memória

No pacote `br.edu.exemplo.repositorio`, crie AlunoRepositorio.

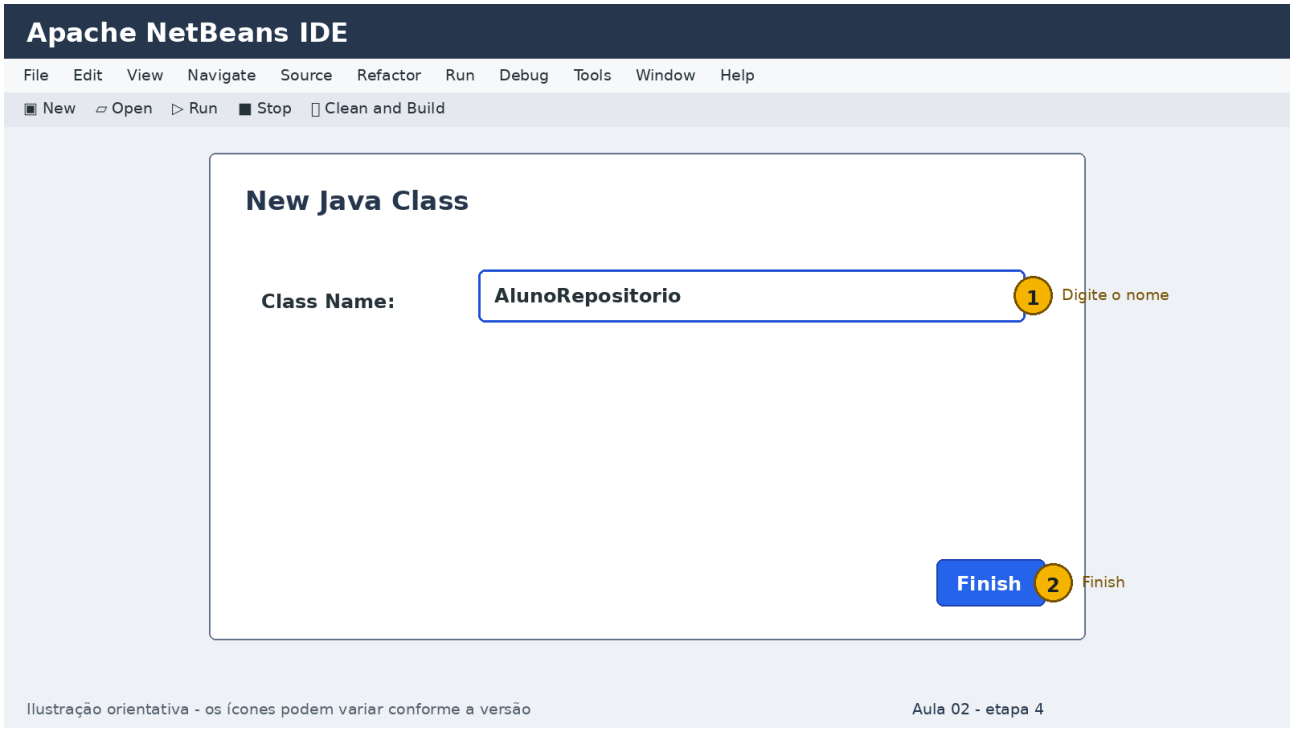


Figura 4 - Criando AlunoRepositorio.java.

Código completo do repositório

```
package br.edu.exemplo.repositorio;

import br.edu.exemplo.modelo.Aluno;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * Repositório provisório em memória.
 * Os dados são perdidos quando o Tomcat é reiniciado.
 */
public class AlunoRepositorio {

    private final Map<Integer, Aluno> alunos = new LinkedHashMap<>();
    private int proximoId = 1;

    public AlunoRepositorio() {
        salvar(new Aluno(0, "Ana Silva", "ana@email.com",
                "Sistemas de Informação", 2, "Noturno"));
        salvar(new Aluno(0, "Carlos Souza", "carlos@email.com",
                "Análise e Desenvolvimento de Sistemas", 4, "Noturno"));
    }

    public synchronized List<Aluno> listar() {
        return new ArrayList<>(alunos.values());
    }

    public synchronized Aluno buscarPorId(int id) {
        return alunos.get(id);
    }

    public synchronized void salvar(Aluno aluno) {
        if (aluno.getId() == 0) {
            aluno.setId(proximoId++);
        }
        alunos.put(aluno.getId(), aluno);
    }

    public synchronized boolean excluir(int id) {
        return alunos.remove(id) != null;
    }
}
```

Entendendo o repositório

Elemento	Função
LinkedHashMap	Mantém alunos associados ao ID e preserva a ordem de inserção.
proximoId	Produz identificadores sequenciais.
listar()	Devolve uma cópia da coleção.
buscarPorId()	Localiza um aluno para edição.
salvar()	Inclui quando ID=0 e atualiza quando o ID já existe.
excluir()	Remove o aluno pelo identificador.
synchronized	Evita alterações simultâneas inconsistentes durante a demonstração.

Os dois alunos criados no construtor são dados iniciais para que a primeira listagem não esteja vazia.

7. Criar AlunoServlet

Em Source Packages, escolha New → Servlet. Use o pacote `br.edu.exemplo.controle`.

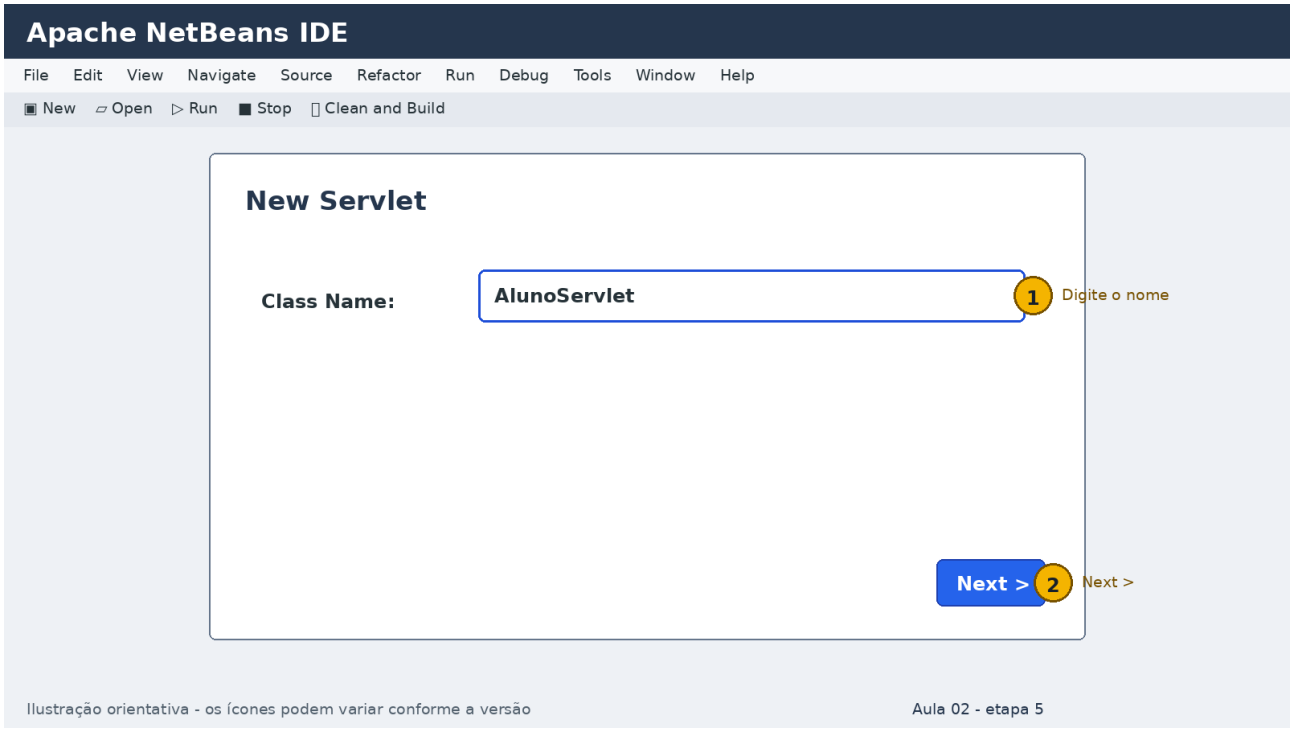


Figura 5 - Criando AlunoServlet.

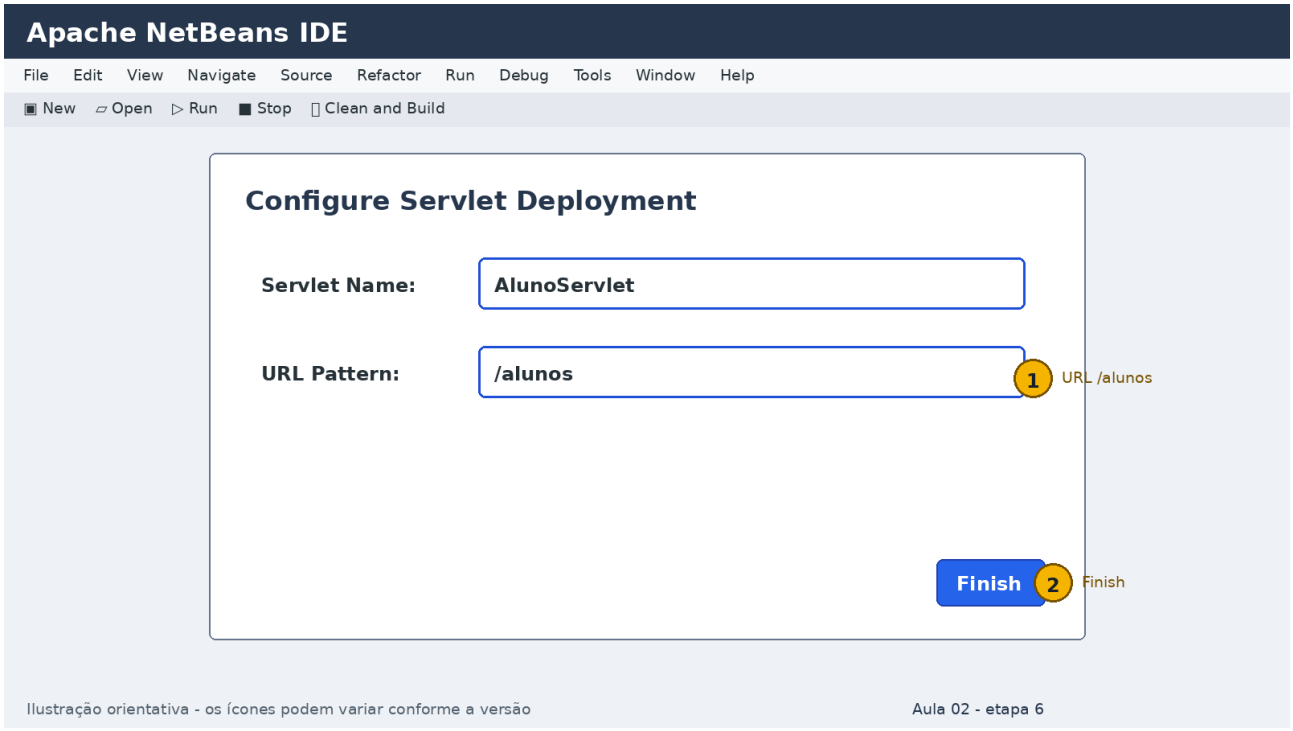


Figura 6 - Associando o Servlet ao endereço /alunos.

A anotação resultante deve ser `@WebServlet("/alunos")`.

Estrutura do Servlet

```
@WebServlet("/alunos")
public class AlunoServlet extends HttpServlet {

    private final AlunoRepositorio repositorio =
            new AlunoRepositorio();

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response) { ... }

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) { ... }
}
```

Método	Atende
doGet	Listagem, abertura do formulário e abertura da edição.
doPost	Salvamento, atualização e exclusão.

Roteamento realizado por doGet

```
String acao = request.getParameter("acao");

if ("novo".equals(acao)) {
    abrirFormulario(request, response, new Aluno());
    return;
}

if ("editar".equals(acao)) {
    abrirEdicao(request, response);
    return;
}

listar(request, response);
```

A comparação é feita com `"novo".equals(acao)`. Dessa maneira, uma ação ausente não provoca NullPointerException.

Roteamento realizado por doPost

```
request.setCharacterEncoding("UTF-8");
String acao = request.getParameter("acao");

if ("excluir".equals(acao)) {
    excluir(request, response);
    return;
}

salvar(request, response);
```

Salvamento e Post/Redirect/Get

```
repositorio.salvar(aluno);

response.sendRedirect(
    request.getContextPath() + "/alunos"
);
```

O redirecionamento cria uma nova requisição GET. Por isso, os atributos do request anterior não são preservados.

Erro de validação com forward

```
request.setAttribute("aluno", aluno);
request.setAttribute("erro", mensagem);

request.getRequestDispatcher(
    "/WEB-INF/views/formulario.jsp"
).forward(request, response);
```

O forward mantém o objeto preenchido e a mensagem de erro na mesma requisição.

Código completo do AlunoServlet - parte 1

```
package br.edu.exemplo.controle;

import br.edu.exemplo.modelo.Aluno;
import br.edu.exemplo.repositorio.AlunoRepositorio;
import java.io.IOException;
import jakarta.servlet.ServletException;
import jakarta.servlet.annotation.WebServlet;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

@WebServlet("/alunos")
public class AlunoServlet extends HttpServlet {

    private final AlunoRepositorio repositorio = new AlunoRepositorio();

    @Override
    protected void doGet(HttpServletRequest request,
                         HttpServletResponse response)
            throws ServletException, IOException {

        String acao = request.getParameter("acao");

        if ("novo".equals(acao)) {
            abrirFormulario(request, response, new Aluno());
            return;
        }

        if ("editar".equals(acao)) {
            abrirEdicao(request, response);
            return;
        }

        listar(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
            throws ServletException, IOException {

        request.setCharacterEncoding("UTF-8");
        String acao = request.getParameter("acao");

        if ("excluir".equals(acao)) {
            excluir(request, response);
            return;
        }

        salvar(request, response);
    }

    private void listar(HttpServletRequest request,
                        HttpServletResponse response)
            throws ServletException, IOException {

        request.setAttribute("alunos", repositorio.listar());
        request.getRequestDispatcher("/WEB-INF/views/lista.jsp")
               .forward(request, response);
    }
```

Código completo do AlunoServlet - parte 2

```
    private void abrirFormulario(HttpServletRequest request,
                                 HttpServletResponse response,
                                 Aluno aluno)
            throws ServletException, IOException {

        request.setAttribute("aluno", aluno);
        request.getRequestDispatcher("/WEB-INF/views/formulario.jsp")
                .forward(request, response);
    }

    private void abrirEdicao(HttpServletRequest request,
                             HttpServletResponse response)
            throws ServletException, IOException {

        try {
            int id = Integer.parseInt(request.getParameter("id"));
            Aluno aluno = repositorio.buscarPorId(id);

            if (aluno == null) {
                request.setAttribute("erro", "Aluno não encontrado.");
                listar(request, response);
                return;
            }

            abrirFormulario(request, response, aluno);
        } catch (NumberFormatException e) {
            request.setAttribute("erro", "Identificador inválido.");
            listar(request, response);
        }
    }

    private void salvar(HttpServletRequest request,
                        HttpServletResponse response)
            throws ServletException, IOException {

        String idTexto = request.getParameter("id");
        String nome = texto(request.getParameter("nome"));
        String email = texto(request.getParameter("email"));
        String curso = texto(request.getParameter("curso"));
        String semestreTexto = request.getParameter("semestre");
        String turno = texto(request.getParameter("turno"));

        int id = 0;
        int semestre = 0;

        try {
            if (idTexto != null && !idTexto.isBlank()) {
                id = Integer.parseInt(idTexto);
            }
            semestre = Integer.parseInt(semestreTexto);
        } catch (NumberFormatException e) {
            encaminharErroFormulario(request, response,
                    new Aluno(id, nome, email, curso, 0, turno),
                    "O semestre deve ser um número inteiro.");
            return;
        }

        Aluno aluno = new Aluno(id, nome, email, curso, semestre, turno);
        String erro = validar(aluno);
```

Código completo do AlunoServlet - parte 3

```
        if (erro != null) {
            encaminharErroFormulario(request, response, aluno, erro);
            return;
        }

        repositorio.salvar(aluno);

        // Padrão Post/Redirect/Get: evita repetição do envio ao atualizar a página.
        response.sendRedirect(request.getContextPath() + "/alunos");
    }

    private void excluir(HttpServletRequest request,
                         HttpServletResponse response)
            throws IOException {

        try {
            int id = Integer.parseInt(request.getParameter("id"));
            repositorio.excluir(id);
        } catch (NumberFormatException e) {
            // O redirecionamento mantém a aplicação utilizável.
        }

        response.sendRedirect(request.getContextPath() + "/alunos");
    }

    private void encaminharErroFormulario(HttpServletRequest request,
                                          HttpServletResponse response,
                                          Aluno aluno,
                                          String mensagem)
            throws ServletException, IOException {

        request.setAttribute("aluno", aluno);
        request.setAttribute("erro", mensagem);
        request.getRequestDispatcher("/WEB-INF/views/formulario.jsp")
               .forward(request, response);
    }

    private String validar(Aluno aluno) {
        if (aluno.getNome().isBlank()) {
            return "O nome deve ser informado.";
        }
        if (aluno.getEmail().isBlank() || !aluno.getEmail().contains("@")) {
            return "Informe um e-mail válido.";
        }
        if (aluno.getCurso().isBlank()) {
            return "Selecione o curso.";
        }
        if (aluno.getSemestre() < 1 || aluno.getSemestre() > 12) {
            return "O semestre deve estar entre 1 e 12.";
        }
        if (aluno.getTurno().isBlank()) {
            return "Selecione o turno.";
        }
        return null;
    }

    private String texto(String valor) {
        return valor == null ? "" : valor.trim();
    }
}
```

8. Organizar as páginas JSP

Dentro de Web Pages → WEB-INF, crie a pasta `views`. Páginas colocadas dentro de WEB-INF não podem ser abertas diretamente pelo navegador; somente o Servlet pode encaminhar a requisição para elas.

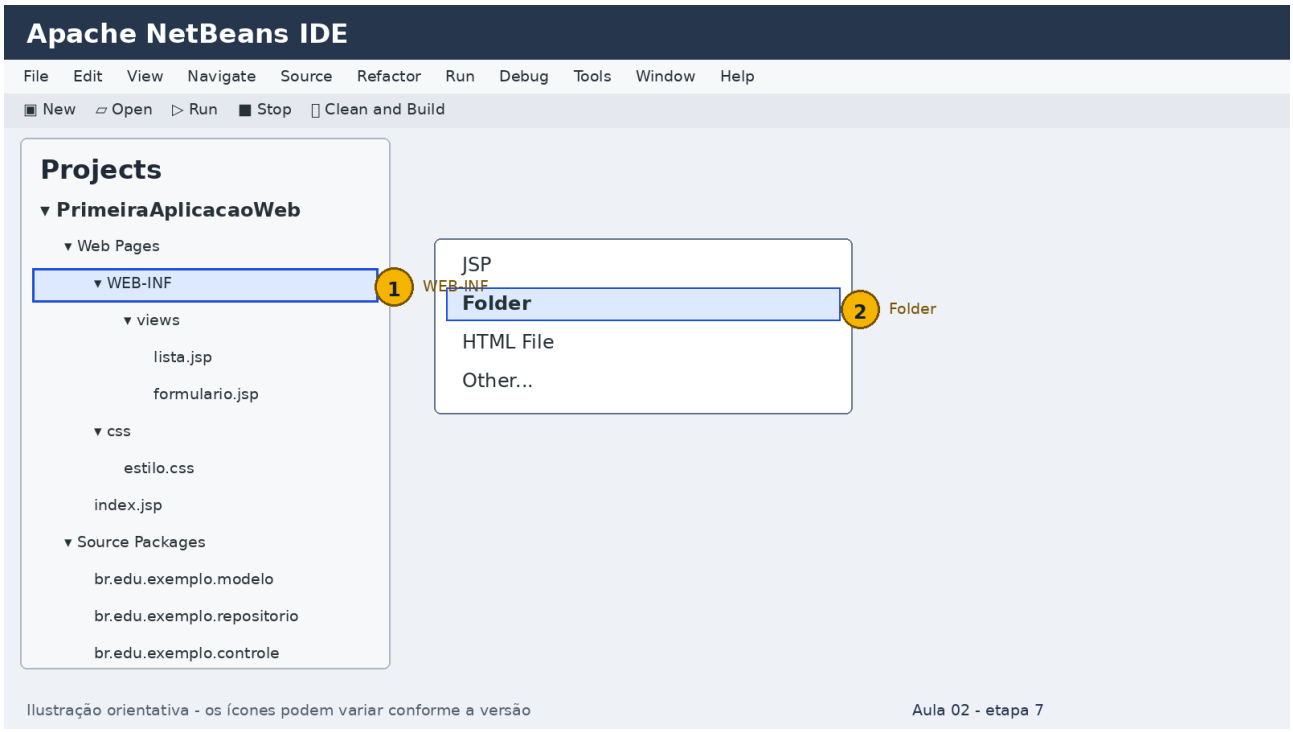


Figura 7 - Criando WEB-INF/views.

Dentro de views, crie `lista.jsp` e `formulario.jsp`.

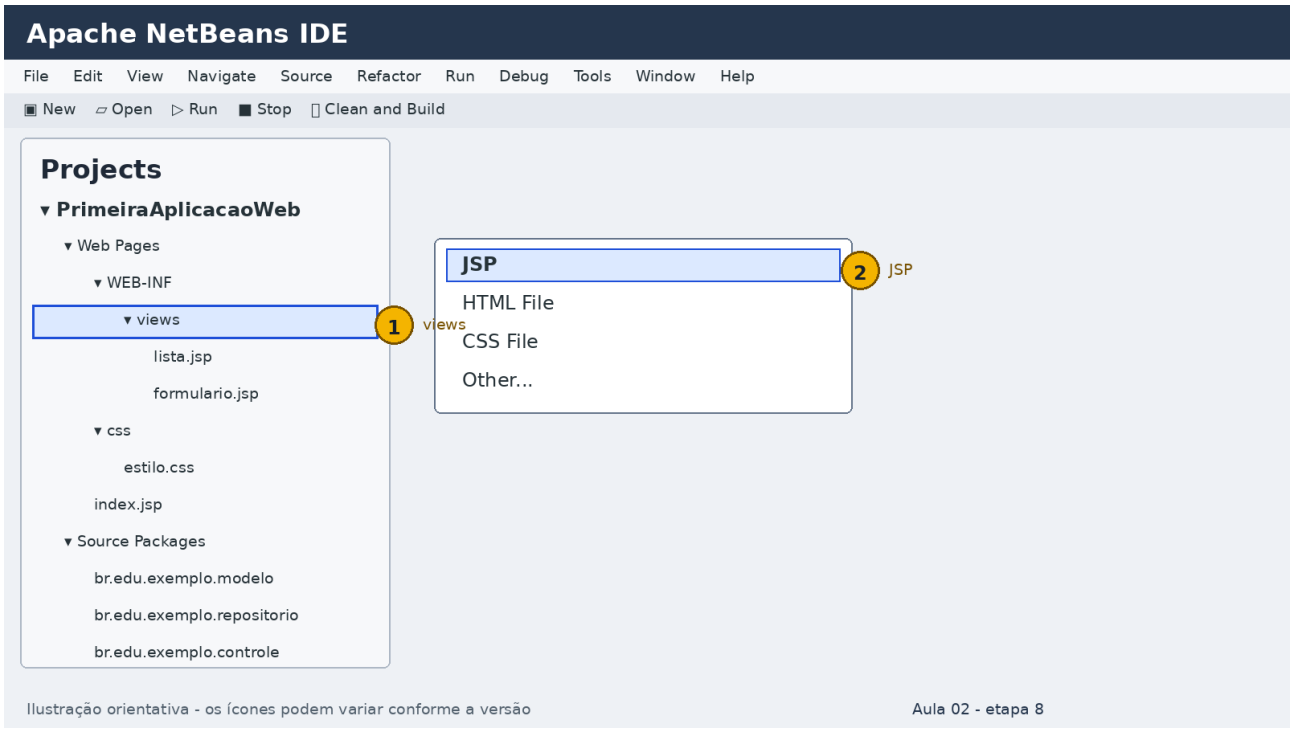


Figura 8 - Criando as páginas JSP.

A listagem recebe o atributo `alunos`, percorre os objetos e apresenta ações de editar e excluir. A exclusão utiliza POST porque altera o estado da aplicação.

Trecho central de lista.jsp

```
<% for (Aluno aluno : alunos) { %>
    <tr>
        <td><%= aluno.getId() %></td>
        <td><%= h(aluno.getNome()) %></td>
        <td><%= h(aluno.getEmail()) %></td>
        <td><%= h(aluno.getCurso()) %></td>
        <td><%= aluno.getSemestre() %>º</td>
        <td><%= h(aluno.getTurno()) %></td>
        <td>
            <a href=".../alunos?acao=editar&id=<%= aluno.getId() %>">
                Editar
            </a>
            <form method="post" action=".../alunos">
                <input type="hidden" name="acao" value="excluir">
                <input type="hidden" name="id" value="<%= aluno.getId() %>">
                <button type="submit">Excluir</button>
            </form>
        </td>
    </tr>
<% } %>
```

Formulário de inclusão e edição

O mesmo formulário serve para duas operações. Quando o objeto tem ID maior que zero, a página apresenta os valores atuais e envia o ID em um campo oculto.

```
<input type="hidden" name="acao" value="salvar">
<input type="hidden" name="id"
       value="<%= aluno.getId() == 0 ? "" : aluno.getId() %>">

<label for="nome">Nome *</label>
<input id="nome" name="nome" type="text" required
       value="<%= h(aluno.getNome()) %>">

<button type="submit">
    <%= editando ? "Salvar alterações" : "Cadastrar" %>
</button>
```

O pacote disponibilizado junto ao tutorial contém os dois arquivos JSP completos.

Observação didática

As JSPs utilizam pequenos blocos Java porque esta etapa não adiciona bibliotecas externas. Em uma evolução posterior, esses blocos poderão ser substituídos por JSTL, mantendo a mesma arquitetura.

9. Adicionar o arquivo CSS

Em Web Pages, crie a pasta `css`. Dentro dela, crie `estilo.css`.

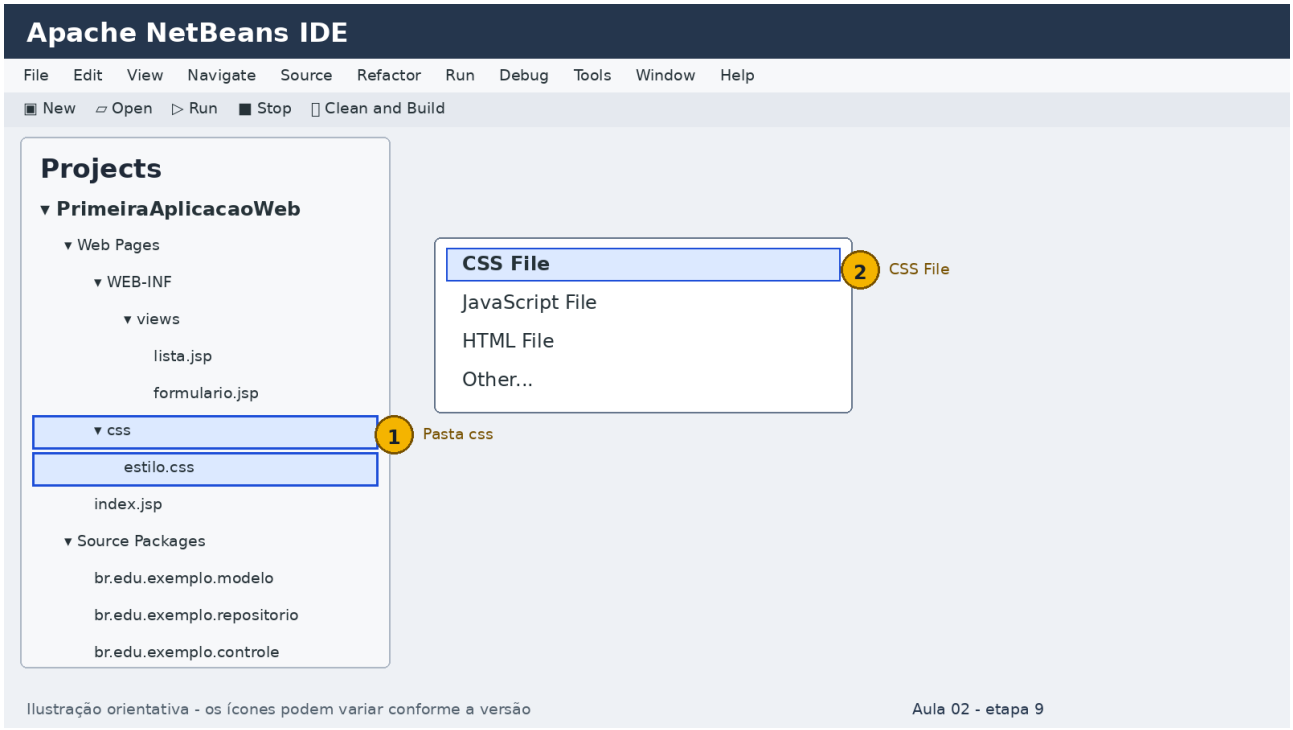


Figura 9 - Criando o arquivo CSS.

As páginas apontam para o CSS usando o contexto da aplicação:

```
<link rel="stylesheet"
      href="<%= request.getContextPath() %>/css/estilo.css">
```

O uso de `getContextPath()` evita endereços fixos e mantém a aplicação funcionando mesmo se o nome do contexto mudar.

10. Estrutura final



Figura 10 - Estrutura esperada após concluir a Aula 02.

Arquivo	Responsabilidade
Aluno.java	Modelo do aluno.
AlunoRepositorio.java	CRUD temporário em memória.
AlunoServlet.java	Controle das requisições.
lista.jsp	Tabela e ações.
formulario.jsp	Inclusão e edição.
index.jsp	Redirecionamento inicial para /alunos.
estilo.css	Apresentação visual e responsividade.
web.xml	Arquivo inicial e versão da aplicação web.

11. Construir e executar

Clique com o botão direito sobre o projeto e escolha Clean and Build. Essa ação recompila as classes e reconstrói o WAR.

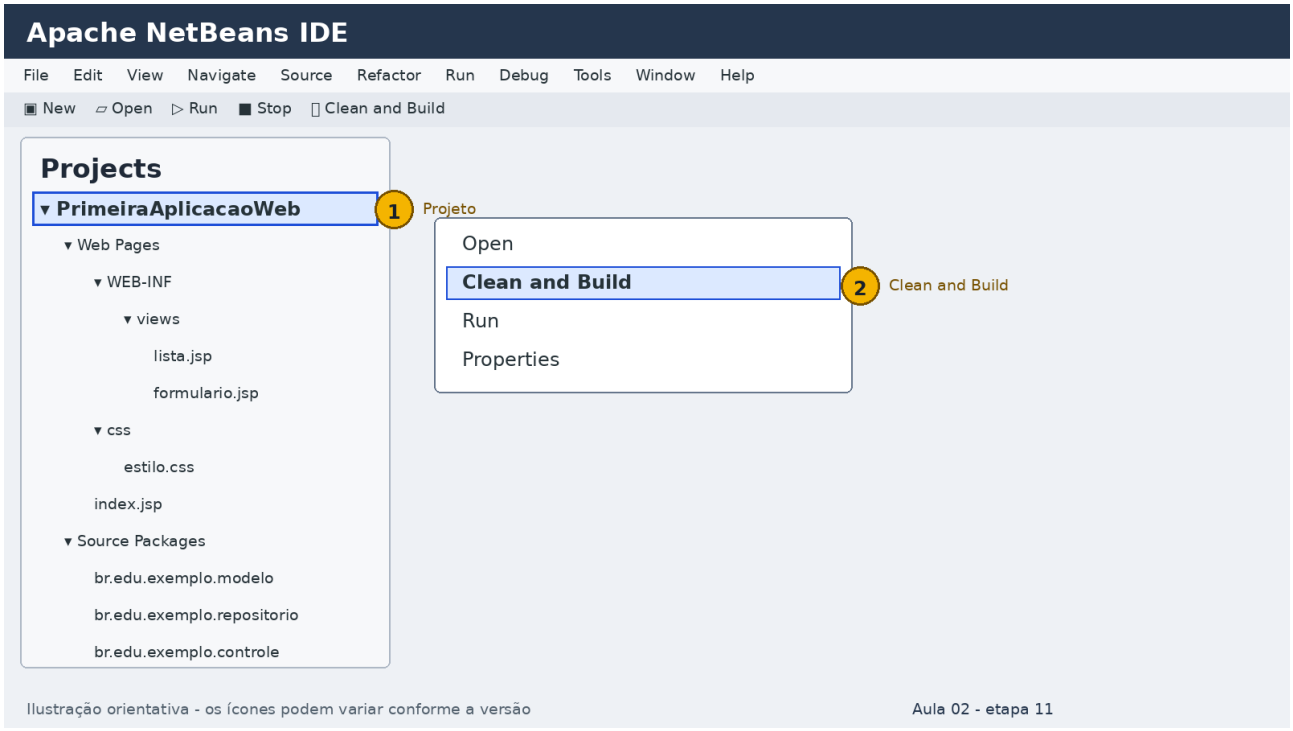


Figura 11 - Executando Clean and Build.

Depois, execute Run Project ou pressione F6.

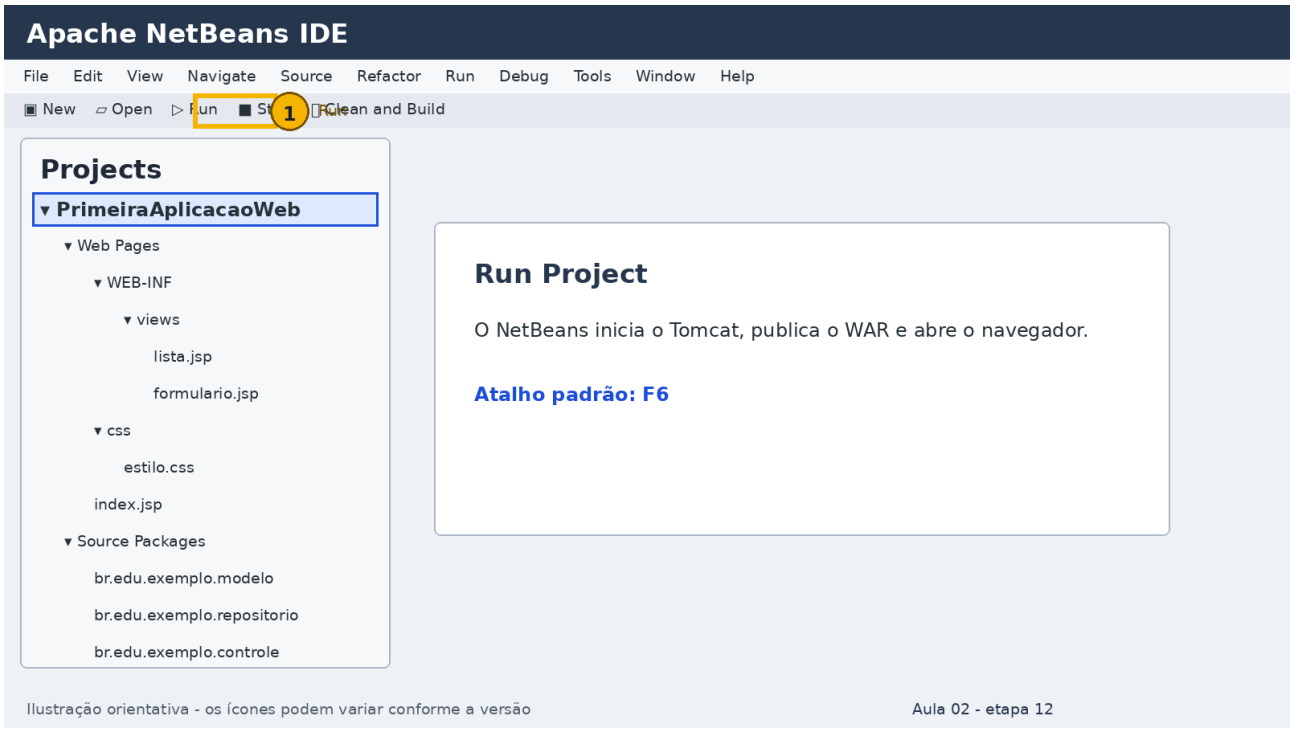


Figura 12 - Executando no Tomcat.

Endereço esperado

```
http://localhost:8080/PrimeiraAplicacaoWeb/
```

12. Testar as funcionalidades

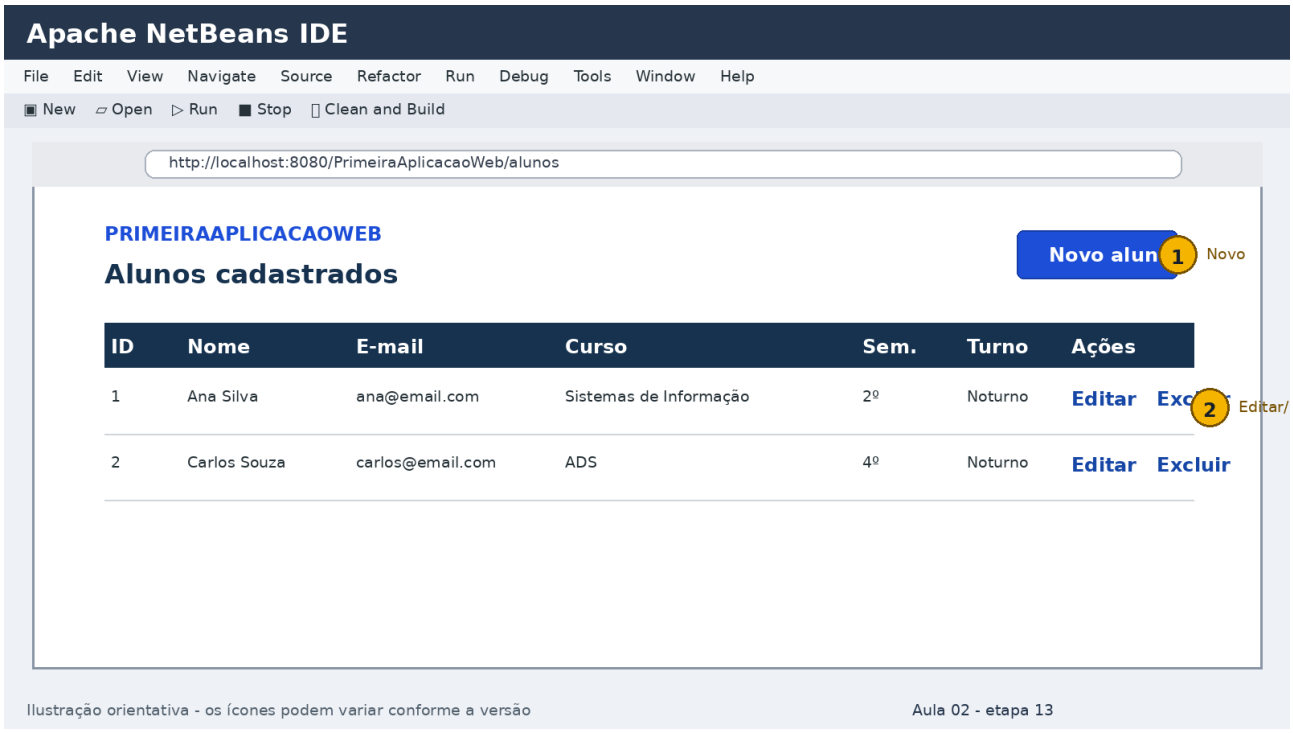


Figura 13 - Tela de listagem de alunos.

Roteiro de teste

- Abrir a listagem.
- Cadastrar um novo aluno.
- Confirmar se ele aparece na tabela.
- Abrir a edição e alterar curso ou semestre.
- Atualizar a página após salvar e verificar que o POST não é repetido.
- Excluir um aluno e confirmar a operação.
- Reiniciar o Tomcat e observar que os dados adicionais desapareceram.

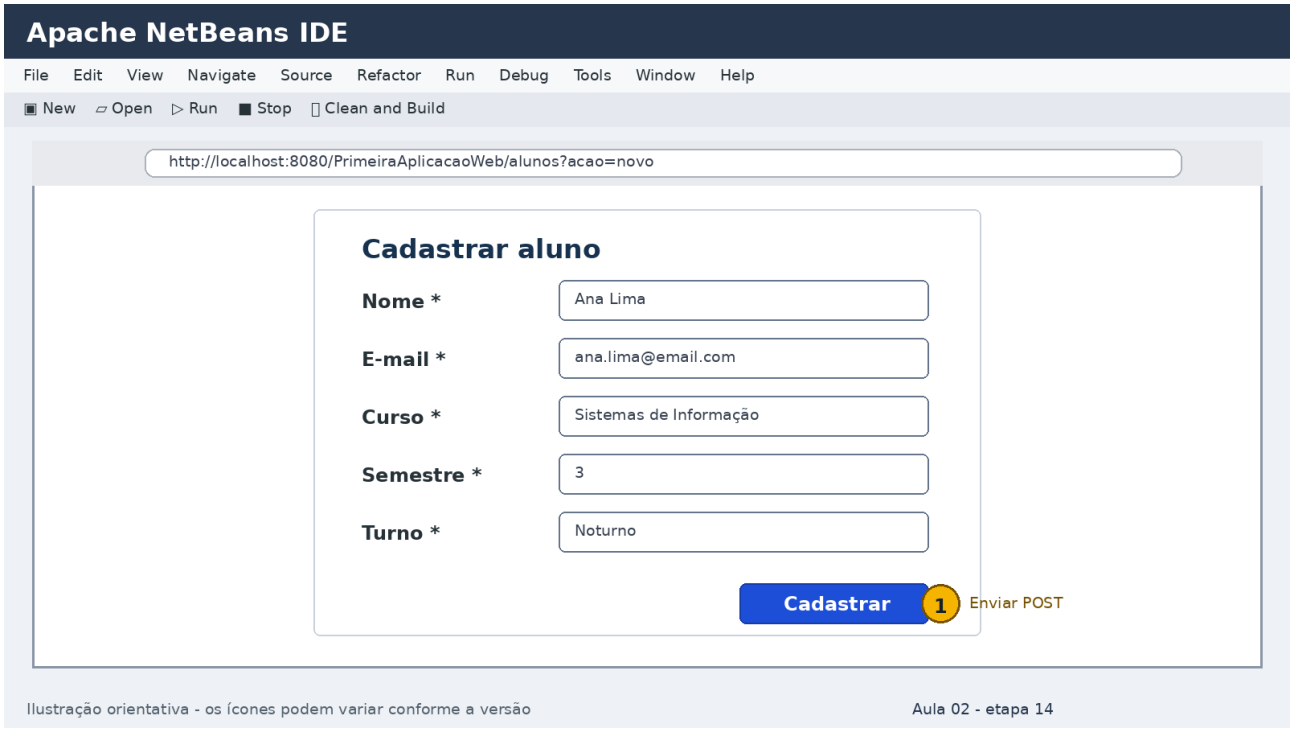


Figura 14 - Formulário usado na inclusão e na edição.

Teste também valores inválidos: nome vazio, e-mail sem @, semestre zero, semestre maior que 12 e turno não selecionado.

13. Erros comuns

Problema	Verificação
404 em /alunos	Confira @WebServlet("/alunos") e se o projeto foi publicado.
JSP abre com 404 diretamente	É esperado: páginas em WEB-INF só são acessadas por forward.
Imports javax.servlet	Use jakarta.servlet com Tomcat 11.
Lista sempre volta aos dois alunos	O projeto foi republicado ou o Tomcat reiniciado; os dados são temporários.
Erro ao converter ID ou semestre	Confira os campos hidden e a validação numérica.
CSS não carrega	Use request.getContextPath() antes de /css/estilo.css.
Caracteres incorretos	Mantenha UTF-8 nas JSPs e request.setCharacterEncoding.
Cadastro duplica ao atualizar	Confirme sendRedirect após um POST válido.

14. Atividade dos alunos

Amplie o sistema acrescentando o campo RA ao cadastro.

Requisitos

- Adicionar o atributo ra à classe Aluno.
- Criar getter e setter.
- Adicionar o campo ao formulário.
- Mostrar o RA na tabela.
- Impedir RA vazio.
- Criar no repositório um método para verificar se o RA já existe.
- Impedir dois alunos com o mesmo RA.
- Preservar o próprio RA durante a edição.

Desafio complementar

Acrescente busca por nome. A URL poderá ser `/alunos?busca=ana`. O repositório deverá retornar somente os alunos cujo nome contenha o texto informado, sem diferenciar letras maiúsculas e minúsculas.

15. Síntese e próxima etapa

Conceito	Aplicação realizada
CRUD	Inclusão, listagem, edição e exclusão.
GET	Consulta e abertura de formulários.
POST	Salvar e excluir.
forward	Exibição das JSPs e preservação dos erros.
redirect	Nova requisição após operações válidas.
WEB-INF	Proteção das páginas de visualização.
Repositório	Camada provisória de armazenamento.
Validação	Regras executadas no servidor.

Fluxo principal

Navegador → AlunoServlet → AlunoRepositorio → AlunoServlet → JSP → navegador

Próxima aula sugerida

Substituir o repositório em memória por persistência real com PostgreSQL e JDBC: criação da tabela, instalação manual do driver no projeto Ant, conexão, DAO, PreparedStatement e tratamento de recursos.

Checklist final

- Projeto continua sem Maven.
- Tomcat 11 e imports jakarta.*.
- Listagem funciona em GET.
- Inclusão, edição e exclusão usam POST quando modificam dados.
- JSPs estão em WEB-INF/views.
- Após POST válido ocorre redirect.
- Dados inválidos retornam com forward e mensagem.
- Clean and Build termina sem erros.